Errand Agent: Design Note

The model proposes. Code approves, executes, and verifies.

State model

Each chat session is in one of nine states: `INTAKE`, `SEARCH`, `PICK`, `HOLD`, `AWAIT_APPROVAL`, `EXECUTE`, `RECONCILE`, `DONE`, `FAILED`. A table in `states.py` lists every legal move; anything else raises `InvalidTransition`. The model never sets state: tools and the executor do. What the user asked for lives in a separate request frame (service, area, date, time, budget) and code works out what is still missing, so the model cannot decide it knows enough. Every run has hard limits of 8 model turns, 12 tool calls (2 kept back for reconciling) and 45 s; hitting one ends the run with a plain message that nothing was booked. Tapping a slot in the provider table skips the model entirely: code builds the approval card. The loose transition table lets a user change their mind from almost any step, at the cost of more paths to test.

Approval enforcement

The model has no `book`, `cancel` or `reschedule` tool. It gets `propose_*` tools that hold a slot and render a card. The card payload (provider, slot, price, fees) is stored with a SHA-256 hash. On Approve, `approve()` checks owner, status, expiry, hash and that the hold is still live, in that order. A typed "yes" takes the same path, matched against a fixed phrase list rather than by the model. Any material change, or a new proposal, voids the open approval and frees its hold, so an old yes cannot be reused for another provider, slot or price. A card expires with its 10-minute hold, and every rejection path releases the hold.

Idempotency and recovery

`begin_attempt` is one Postgres function: it flips the approval to `consumed` and writes the attempt row in one transaction (write-ahead), so a double tap or replay finds nothing to consume. The key is `idem_<approval id>`, so every retry carries the same key and the provider replays its stored answer. After a timeout the executor calls `get_booking_status` first, retries with the same key at most twice and only while the hold is live, and says "booked" only once status confirms it; otherwise the attempt stays `unknown` and the user is told so. On start, `recover()` closes stale runs and resumes every attempt left `started` or `unknown`; the same sweep runs on the user's next message. A per-user lock stops two requests racing.

Memory

Each user message goes through a separate small model call that proposes facts *with the user's own words as evidence*; code then saves only what passes: a known key, a valid value, evidence that really appears in the user's message (so a review or provider can never become a "preference"), provider names resolved to real ids, and never on a "forget" message. The chat model is told exactly what was saved, so it cannot claim more. Explicit facts are active at once; inferred ones (from repeated bookings) wait as `pending_confirmation` and are hidden from the model until the user agrees; forgotten ones leave a tombstone with the value wiped. Only active facts enter the 6,000-token context, ranked by relevance and capped. Forgetting is thorough: it also redacts chat history, drops summaries and scrubs open requests that quote the fact. Cost: one extra model call (~1 s) per message. Known gap: the request frame takes the model's word for what the user said; the `_from_memory` tag only covers fields that code filled.

Observability

Every run writes ordered spans (`state_transition`, `model_call`, `tool_call`, `context`, `approval`) with arguments, sanitized results, latency, token counts and an estimated cost, and the same run is one LangSmith trace (each model and tool call a step, each chat a thread, a booking score on approve runs). Prompt text stays out of our database. The context span lists the memory, summary and message ids sent each turn, which is how tests prove a forgotten fact never reached the model. Excerpt from a real run (real model, Tuesday slot: `book` times out *after* succeeding), the Approve run, which has no model call:

```
approval: approved          hash ok, approval consumed, attempt row written first (write-ahead)
state: AWAIT_APPROVAL -> EXECUTE
exec.book  idem_bff735fd..  ERROR ToolTimeout   outcome unknown: no blind retry, no claim of success
state: EXECUTE -> RECONCILE
exec.get_booking_status     booked bk_6d4fd50d, Rs 1250   asked the source of truth (by hold id, then by key)
state: RECONCILE -> DONE    "Booked and verified: ..."   retry_count 0, 1 booking, 1 attempt  (raw: docs/traces/)
```

Storage: Postgres through a pooled direct connection, reads cached while a request holds the user lock (49 -> 19 database calls per turn). Checks: 61 offline eval scenarios against a scripted model with the network blocked, and 15 injected bugs, each of which must fail at least one scenario (15/15 caught).